

The agent said the
tenant was **ready**.
It wasn't.

The Evaluation Gap — why you can't ship
judgment you can't measure, and how to measure
it.

Swipe →

A confident report that was simply wrong.

The agent assessed the tenant, wrote a clean readiness summary, and marked it **green**. Half the accounts were never actually verified.

Nothing crashed. No stack trace. Just a **fluent, plausible judgment** that nobody caught — until it reached the customer.

A broken action crashes.
A broken judgment **lies**
fluently.

Reliability engineering catches the first. It does nothing for the second.

The moment an agent *decides* rather than just *executes*, correctness stops being a runtime property — it becomes an **evaluation problem.**

Fluency reads as correctness.

Polished output disarms reviewers. A well-formatted, confident answer gets **rubber-stamped** — the better the prose, the less it's scrutinized.

Your evaluation has to reward **being right**, not **sounding right**. Those are different signals, and only one of them ships safely.

Score the trajectory, not just the destination.

Two agents reach “ready.” One **verified** every account; one **guessed**. Same answer — very different systems.



Grade the **path**: did it check what it claimed to check? Outcome-only metrics can't tell luck from judgment.

Offline gate. Online guardrail.

Offline gate

- Golden set of hard cases
- Runs in CI, blocks release
- k-run consistency check

Online guardrail

- Canary on live traffic
- Watch escalation rate
- Auto-rollback on drift

One proves it's good **before** ship. One proves it's **still** good after.

Useful tool. Not an oracle.

- **Calibrate** it against human labels before you trust a single score.
- **Distill** its agreement into cheaper, faster checks where you can.
- **Backstop** with hard rules for anything safety- or money-critical.

A judge that shares the generator's blind spots will **confidently co-sign the same mistake.**

An evaluation-readiness checklist — and a confidence threshold.

Do you score the **trajectory**? Is there an **offline gate** in CI and an **online guardrail**?
Below the threshold — does it **escalate** instead of guess?

Full post + checklist — link in the comments.
Part 2 covered reliability. Follow for Part 4:
Accountability & Auditability.